

WORK INTEGRATED LEARNING, OCTOBER 2026

Toolshed Hire, Task 2

The built system for INSY7315

- Teon Kleynhans, ST10434209, working alone
- Live at toolshed-hire.vercel.app



TOOLSHED
HIRE

Rental management for a tool hire business
with three branches in Cape Town

The problem

3

branches

400

tagged units

120

models



Bookings live in a paper diary and a WhatsApp group

A tool hire business with three branches in Cape Town

1

The same unit is promised twice

The database itself now refuses a double booking

2

Staff phone each other to find stock

The locator finds any unit at any branch by its tag

3

The owner cannot say which equipment earns its keep

The report now gives utilisation and gross contribution



What Task 2 delivers



Customer journey

From the catalogue to a confirmed booking

01 02 03 04 05 06 07 08 09

SC-01 to SC-09, on the API



Counter journey

From a walk-in to checkout, return, damage and deposit settlement

10 11 12 13 14 15 16 17 18

SC-10 to SC-18, on the API



The owner's side

The dashboard, the report with its CSV, the catalogue and prices, the asset register, users and customer holds, the audit trail and charge corrections

19 20 21 22 23 24

SC-19 to SC-24, on the API

24 of 24

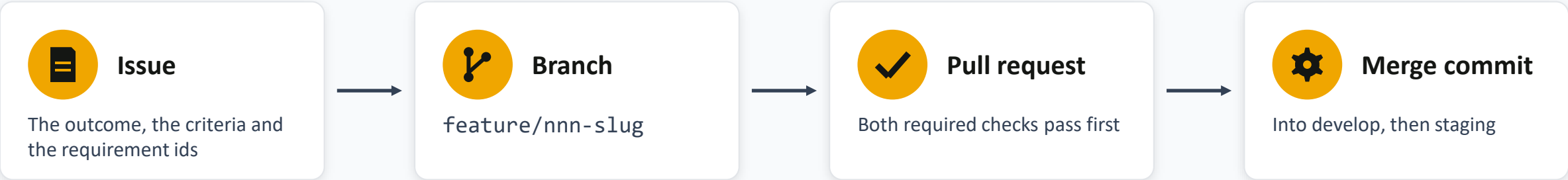
screens read from the API, and no screen shows sample data any more



Two environments

Deployed by the pipeline only, and live as v0.4.1

Requirements and how they are traced



28 functional requirements

27 built

1 partly built, FR-28, whose statement download is planned

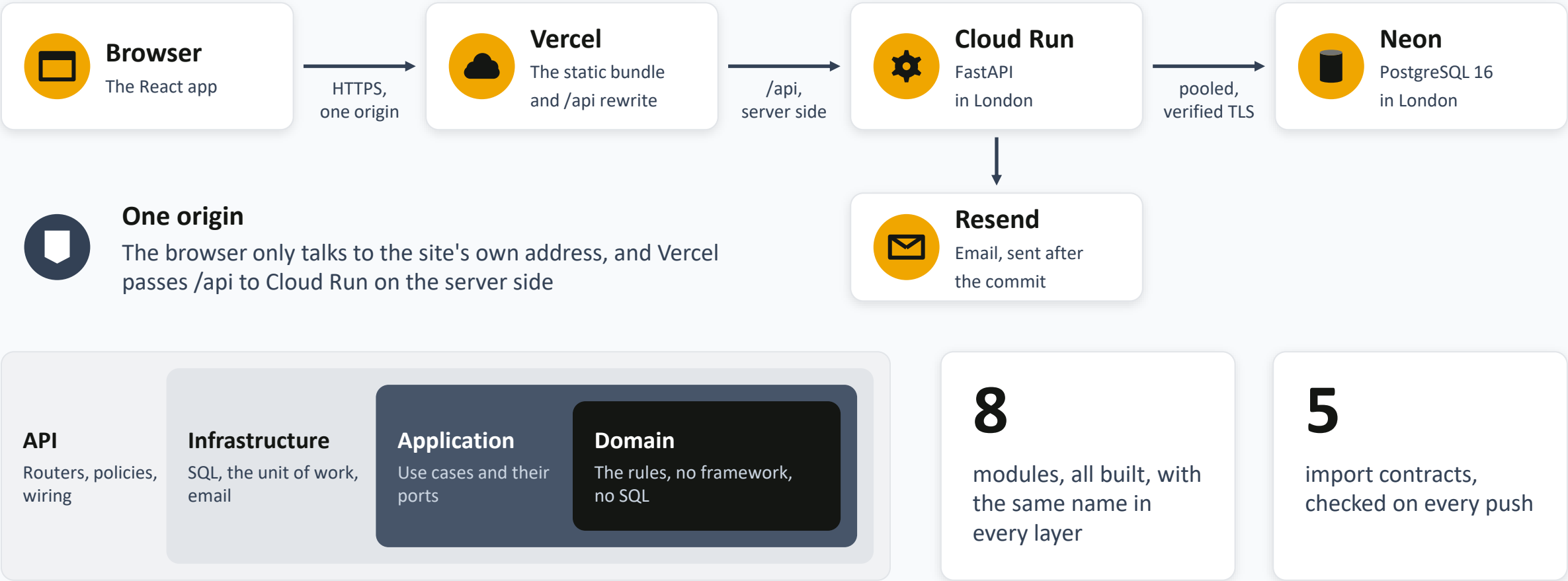
18 non-functional requirements, each mapped to how I verify it

FR-07, followed through

No unit ever holds two overlapping bookings

Status	Built
Pull requests	#24 on branch 002, #38 on branch 005
Tests	test_exclusion_constraint.py test_concurrent_holds.py

Architecture

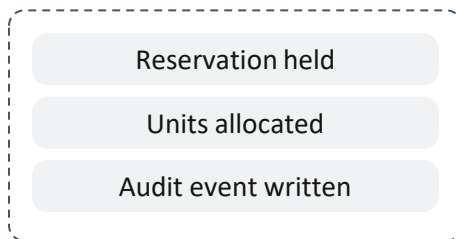


The four design patterns



Repository with Unit of Work

One transaction for a hold, its allocations and its audit event.

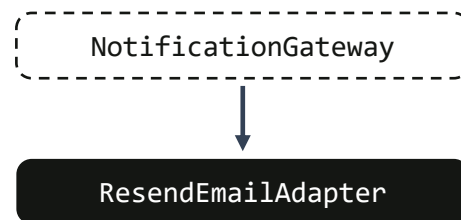


application/unit_of_work.py



Adapter

The email provider sits behind a port, with a transactional outbox.

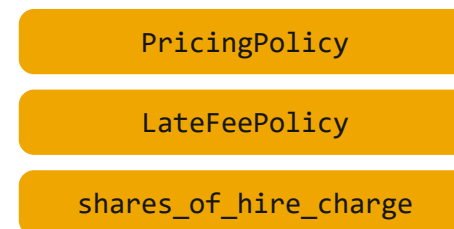


infrastructure/notification



Strategy

The pricing and late fee policies, and the report's share of a hire charge.



domain/policies



State

Eight reservation states. A move is refused unless the state allows it.



domain/states



The database

17

tables with singular names

12

hand written migrations



Indexes proved in use

Each later migration adds indexes or one column, and tests prove each index is used



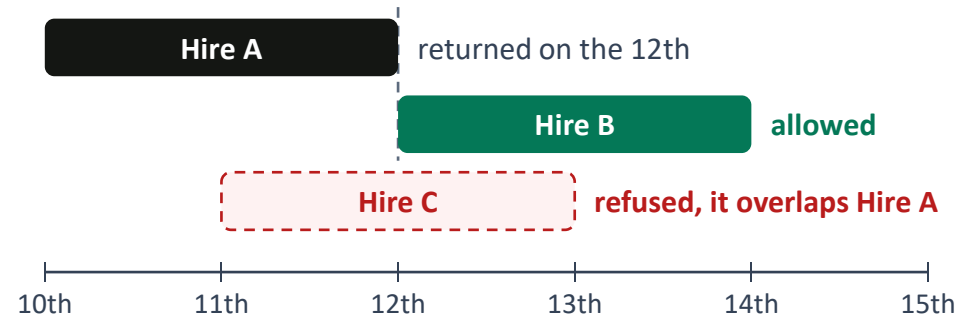
Two database roles

The running application cannot change the schema or rewrite the audit trail

THE CONSTRAINT

```
ALTER TABLE asset_allocation
ADD CONSTRAINT asset_allocation_no_overlap
EXCLUDE USING gist (
    asset_id WITH =,
    daterange(start_date, end_date, '[')') WITH &&
) WHERE (released_at IS NULL);
```

One unit, half open periods



Security



Sessions

A fifteen minute token in memory and a rotating refresh cookie



Deny by default

Ownership answered as not found, branch scope for counter staff, and the owner's functions for the administrator alone

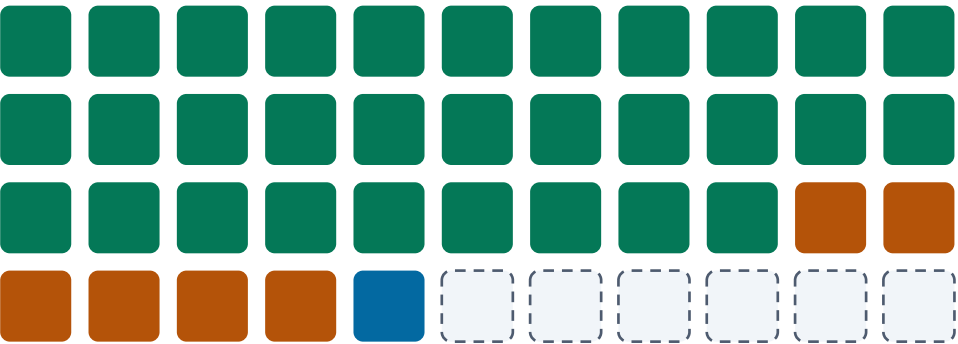


Lockout and throttles

Security headers checked on the deployed site after every deployment, and no secret in the repository

44 controls from the design

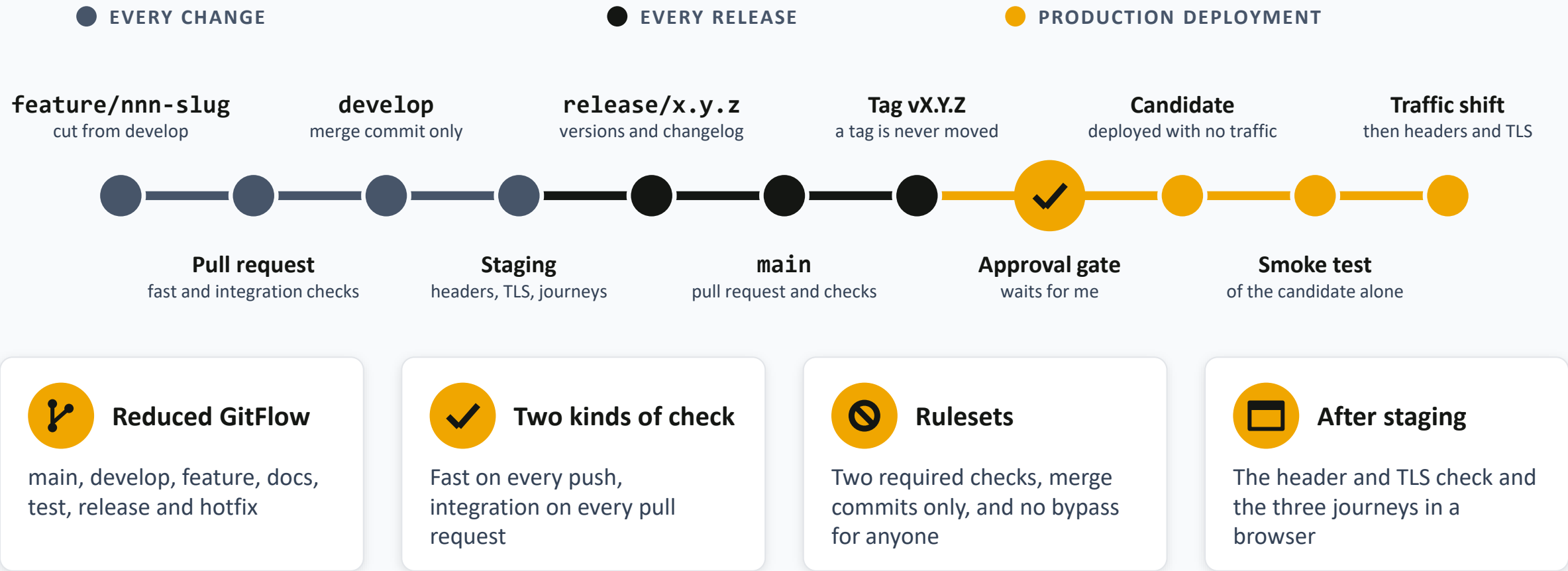
Each with its status and the file, test or step that shows it



- Green 31 in place
- Orange 6 partly in place
- Blue 1 replaced
- Dashed 6 planned

Still open: the photograph upload, edge filtering, alerting and the incident procedure

Pipeline and branching, a live look at GitHub



Hosting and the release procedure



Three hosts

Vercel for the site, Cloud Run in London for the API, Neon in London for the database



Keyless deployment

Workload Identity Federation, and every secret kept in Secret Manager



My approval, then a candidate

A release tag waits for my approval, a candidate is smoke tested before any traffic, and headers and TLS are checked last



One instance at most

With a budget alert, because this runs on a free trial

	Staging	Production
Deployed by	A merge into develop	A version tag on main, after my approval
Database	Neon branch staging	Neon branch main
New revision	Takes traffic at once	No traffic until its smoke test passes
Checked after it	Smoke test, headers, TLS, three journeys	Smoke test, headers and TLS



The API scales to zero, so the first request after a quiet spell waits for it to wake

Live demonstration

- 1

The customer journey on production

Search, hold, confirm, cancel and the hire history, about one and a quarter minutes
- 2

The counter journey

A walk-in, checkout, the return with its deposit and the locator, about one and three quarter minutes
- 3

The worked example from Task 1

Rental TSH-H-26-000098, opened on the return screen
- 4

The owner's side

From the dashboard and the report to prices, units, staff, holds and the audit trail, about two and a half minutes

THE WORKED EXAMPLE, ON PRODUCTION

TSH-H-26-000098

SETTLED

Deposit held	R1,200.00
Late fee, two days at R120.00	R240.00
Withheld from the deposit, R208.70 plus R31.30 VAT	
Released to the customer	R960.00
Due at the counter	R0.00

A test follows these figures through the API on every pull request.



Testing

4,623

backend tests on
PostgreSQL 16

100%

of lines in domain and
application, floor 70

1,455

frontend unit and
component tests

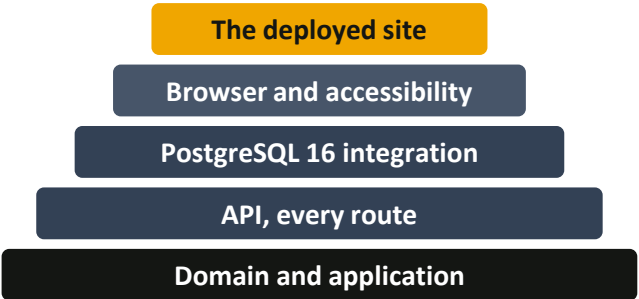
392

browser and
accessibility tests on
the real API

109

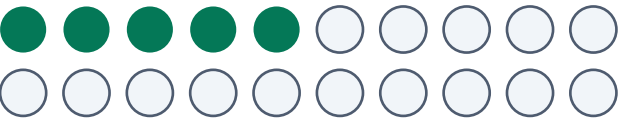
axe scans of all 24
screens, zero findings

Five kinds of test



mypy strict on 341 files, and 5 layer import contracts

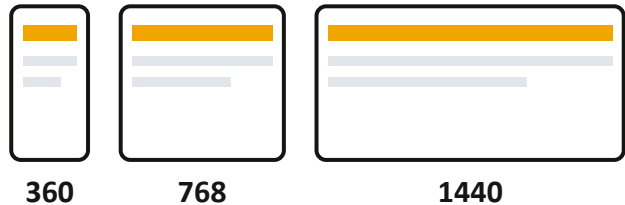
Twenty holds, five units



5 holds and **15 clean refusals**
No unit is ever held twice

Every pull request, against PostgreSQL 16

Every screen, three widths



Pixels wide, and nothing scrolls sideways



Deviations from the design

THE DESIGN

Production capped at four instances

Email delivered to every customer

A later correction to a settled hire is a reversing charge

Utilisation worked out in one query with window functions

WHAT I BUILT

→ **One instance in both environments**

→ **Restricted mode, delivered to my own address only**

→ **A settled hire only takes a correction that gives money back**

→ **The report works out its days in the domain**

WHY

An instance cap is the only hard limit on cost while this runs on a free trial.

There is no verified sending domain, and a stranger should never get mail from a university project.

Asking a settled hire for more would reopen it. Money found owed later is a matter for the office.

Each definition has one copy, tested with no database, and it takes a few milliseconds at 400 units.



What is left for Task 3

All 22 planned branches merged

v0.1.0

v0.2.0

v0.3.0

v0.4.0

v0.4.1

task2-v1.0

v1.0.0 in Task 3

001

002

003

004

005

006

007

008

009

010

011

012

013

014

015

016

017

018

019

020

021

022



A customer's own data

The statement download, data export and the retention period



Damage photographs

The upload with its controls, and edge filtering



Measurements and drills

Load, volume and cold start, the restore drill, usability sessions



Alerting and incidents

Alerts on the logged events, and the incident procedure



Release v1.0.0

With the report and the user guide, once the system matches the whole design

Thank you



toolshed-hire.vercel.app



github.com/teon-artilionAI/toolshed-hire



Teon Kleynhans, ST10434209

The README is the place to start

